



**Universidad
Nacional de
General
Sarmiento**

Trabajo Práctico

Programación 1 - Primer Semestre 2026

COMISION 1

GRUPO 6

INTEGRANTES:

• **Lautaro José Ferreyra**

DNI: 46.421.509

email: ferreiralautaro268@gmail.com

• **Gabriel Fernando Fournier**

DNI: 47.574.818

email: gabrielferfour@gmail.com

• **Bautista Gonzalez Ferrere**

DNI: 47.883.028

email: gonzalezferrerebautista@gmail.com

Introducción

El presente trabajo práctico consiste en el desarrollo de un videojuego de plataformas en Java inspirado en Mario Bros. El objetivo del juego es controlar a la protagonista, una princesa, a través de un escenario compuesto por distintas plataformas flotantes, evitando enemigos y avanzando hasta alcanzar el castillo ubicado al final del mapa.

Durante el desarrollo se implementaron mecánicas de movimiento, salto, disparo de proyectiles, detección de colisiones, sistema de vidas, generación de enemigos, generación de islas, aparición de ítems en el mapa, puntaje y condiciones de victoria y derrota. El trabajo permitió aplicar conceptos fundamentales de programación orientada a objetos, diseño de clases y modelado de entidades dentro de un videojuego.

Descripción e Implementación

Clase Juego

Es la clase principal del proyecto y coordina todo el funcionamiento del juego.

Variables de instancia principales

- Entorno entorno: administra la ventana y los eventos del juego.
- Personaje per: representa al personaje del jugador.
- Proyectil proyectil: almacena el disparo activo.
- Explosión explosión: controla el efecto visual al impactar un proyectil.
- Fondo fon: representa el escenario de fondo.
- Isla[][] islas: almacena todas las plataformas del nivel.
- Enemigo[][] enemigos: almacena los enemigos presentes en cada nivel.
- Castillo castillo: objetivo final del juego.
- Vidas[] corazones: muestra gráficamente las vidas restantes.
- Puntaje puntaje: controla el sistema de puntuación.
- Items[] items: contiene los ítems que aparecen en el mapa.

Métodos principales

- generarMapa(): genera las islas en el mapa. Estas se distribuyen en 4 niveles de 24 islas, se generan de manera aleatoria con una separación que varía. El ultimo nivel es el piso, el cual tiene tamaño y separación fijo, y solo tiene 19 islas.
- dibujarIslas(): dibuja las plataformas ya generadas e incluidas en la matriz islas.
- dibujarCorazones(): muestra las vidas del jugador arriba a la derecha de la interfaz.
- limite(): controla el desplazamiento del escenario completo a medida que el jugador se mueve.

- `caidaAlVacio()`: detecta si el personaje cae fuera del mapa y si es así le resta una vida.
- `gano()`: verifica la condición de victoria.
- `perdio()`: verifica la condición de derrota.
- `reiniciarJuego()`: se encarga de reestablecer las variables para que el juego pueda ser reiniciado sin necesidad de correr nuevamente el programa.
- `spawnearEnemigo()`: es utilizada por el tick para spawnear enemigos.

La clase `Juego` centraliza toda la lógica del juego: movimiento, colisiones, disparos, enemigos, puntaje, vidas y condiciones de victoria o derrota.

Clase Personaje

Representa al jugador controlado por el usuario.

Variables principales

- Posición (x, y).
- Dirección del personaje (dirección).
- Imágenes para ambos sentidos de movimiento.
- Cantidad de vidas.
- Estado de salto y contador de tiempo de salto.
- Límites de colisión y tamaños de imagen.
- Estado de invulnerabilidad y contador de tiempo de invulnerabilidad.

Métodos principales

- `dibujar()`: muestra el personaje en pantalla.
- `moverX()` y `moverY()`: modifican su posición en los ejes.
- `caer()`: aplica gravedad (es llamada por el tick constantemente mientras el jugador no esté tocando el piso).
- `iniciarSalto()`: comienza un salto, es llamada cuando el jugador toca la tecla arriba.
- `salto()`: ejecuta el movimiento ascendente.
- `cortarSalto()`: permite realizar saltos de distinta altura al cortarlo si el jugador deja de pulsar la tecla arriba.
- `actualizarBordes()`: recalcula los límites de colisión y es llamada constantemente.
- `perderVida()`: reduce una vida.
- `iniciarInvulnerabilidad()`: inicia invulnerabilidad en el personaje por un tiempo determinado.

- `restarTiempoInvulnerabilidad()`: es llamada constantemente y determina los tiempos de la invulnerabilidad.

Métodos de colisión

Colisiones con islas:

- `seApoyaEn()`
- `chocaCabezaCon()`
- `chocaPorDerechaCon()`
- `chocaPorIzquierdaCon()`

Enemigo o ítem:

- `colisionCon()`

Estos métodos permiten detectar interacciones con plataformas, enemigos e ítems.

Clase Enemigo

Es el objeto que representa un enemigo del jugador. Dispone la dificultad al juego al evitar que el personaje pasee libremente por el mapa.

Variables principales

- Posición (x, y, escala).
- Dirección de movimiento.
- Imágenes para cada sentido y para la animación de muerte del personaje.
- Estado “muerto”, que determina si el enemigo ya murió para que el tick lo pueda eliminar, y “muriendo”, que determina si se está ejecutando la animación de muerte.
- Tiempos de la animación de muerte.
- Límites de colisión con personaje e islas.

Métodos principales

- `dibujar()`: muestra el enemigo.
- `mover()`: realiza el desplazamiento horizontal al lado contrario de la aparición del enemigo.
- `morir()`: inicia la secuencia de eliminación y si es llamado nuevamente mata al enemigo por completo.
- `muriendo()`: ejecuta la animación de muerte, es llamado constantemente por el tick.
- `actualizarBordes()`: actualiza los límites de colisión.
- `fueraDePantalla()`: verifica si abandonó el escenario para que el juego pueda eliminar el objeto.

- `colisionaCon(Isla)`: verifica si colisiono con una isla.

Clase Proyectil

Representa el proyectil lanzado por el jugador al hacer clic, y permite al jugador eliminar enemigos a distancia.

Variables principales

- Posición (x, y, escala).
- Velocidad horizontal y vertical.
- Imagen.
- Ángulo de movimiento (se calcula según donde se realiza el clic).
- Radio de colisión (bordes “artificiales”, delimitados en radio de 10).

Métodos principales

- `mover()`: actualiza la posición del proyectil.
- `dibujar()`: muestra el proyectil.
- `fueraDePantalla()`: detecta si esta fuera de la pantalla.
- `colisionaCon(Isla)`: detecta si colisiono con una plataforma.
- `colisionaCon(Enemigo)`: detecta impacto con algún enemigo.

Clase Explosión

Representa el efecto visual generado cuando un proyectil impacta.

Variables principales

- Posición (x, y, escala).
- Imagen.
- Tiempo de duración.

Métodos principales

- `dibujar()`: muestra la explosión.
- `actualizar()`: reduce su tiempo de vida.
- `terminada()`: indica cuándo debe eliminarse.

Clase Isla

Representa las plataformas sobre las que se desplaza el personaje. El constructor verifica el tamaño ingresado y selecciona una de las cuatro imágenes cargadas para

representar al tamaño de la isla. En conjunto forman el escenario principal donde ocurren los desplazamientos del personaje.

Variables principales

- Posición (x, y, escala).
- Tamaño (ancho, largo, tamaño).
- Imagen.
- Límites de colisión (bordes).

Métodos principales

- dibujar(): muestra la plataforma.
- actualizarBordes(): recalcula los límites.

Clase Castillo

Representa el castillo posicionado al final del mapa en el cual el personaje ingresa para ganar la partida. Este se genera en la última isla del piso, que sería el ultimo nivel de islas.

Variables principales

- Posición (x, y).
- Imagen.
- Escala (escala, ancho, alto).
- Límites de colisión (bordes).

Métodos principales

- dibujar(): muestra el castillo.
- moverX(): desplaza el castillo junto con la cámara.

Clase Fondo

Representa la imagen de fondo del escenario.

Variables principales

- Posición (x,y).
- Dimensiones (ancho, alto, escala, que varían según la imagen cargada).
- Imagen.

Métodos principales

- dibujar(): muestra el fondo.
- moverX() y moverY(): desplazan el escenario cuando la cámara se mueve.

Clase Vidas

Representa las vidas del personaje con la finalidad de graficarlas en la pantalla en forma de corazones.

Variables principales

- Posición (x, y).
- Imagen de corazón normal.
- Imagen de corazón roto.
- Estado de vida.

Métodos principales

- dibujar(): muestra el corazón correspondiente según su estado.

Clase Puntaje

Administra el sistema de puntuación.

Variables principales

- Puntaje acumulado.
- Estadísticas (kills, mejorRacha, racha).
- Multiplicador (basado en la racha de kills del jugador).
- Modificadores de puntos para mostrar en forma de animación al sumar o restar puntos.

Métodos principales

- sumar(): agrega puntos.
- restar(): descuenta puntos.
- kill(): otorga puntos al eliminar un enemigo.
- pierdeVida(): resta puntos al perder una vida.
- resetMultiplicador(): reinicia el multiplicador.
- dibujar(): muestra el puntaje actual, la racha, el multiplicador y las sumas y restas constantes de puntos.
- mostrarStats(): muestra estadísticas finales, se utiliza al finalizar el juego.

Problemas encontrados y soluciones

Detección de colisiones

Uno de los principales desafíos fue lograr colisiones precisas entre el personaje, los enemigos, los proyectiles y las plataformas, especialmente entre la parte inferior del personaje y la superior de las islas. Para resolverlo se implementó una actualización constante de los bordes al modificar la posición de cualquier sea el elemento, además de modificar la caída para que detecte que “toco piso” a pesar de que este a unos 8 pixeles de la isla, para evitar cualquier traspaso fantasma. A esta distancia la redujimos con un teletransporte que no se puede notar.

Además, con la colisión entre personaje y enemigo, tuvimos un error en los for que recorrían a la matriz de enemigos y por este mismo algunos enemigos no le quitaban una vida al jugador. Este problema lo solucionamos en la reestructuración del código.

El salto

Otro problema que tuvimos fue que el salto parecía poco natural y tenía muchos errores, al principio volaba, y no nos queríamos quedar con el salto simple. Al final pudimos crear una función que corta el salto y se llama cuando el jugador deja de tocar la flecha arriba. De esta forma, logramos que el salto se vea como en los juegos retro, que al mantener el salto lograba su altura máxima, mientras que, al simplemente pulsar sin mantener, el salto es más corto.

Generación de islas

Cambiamos reiteradas veces la generación de islas porque nos parecía antinatural y muy extraña. Logramos que se generen de manera aleatoria y con distancias naturales, distintos tamaños y con los tamaños y distancias fijos en el piso como pide la consigna. La distancia de las islas es la justa y necesaria para que el salto se pueda realizar de nivel a nivel.

Conclusión

En conclusión, este proyecto fue extremadamente útil para comprender la lógica de la programación orientada a objetos, y a su vez pulir conocimientos ya aprendidos en IP y en el comienzo de Programación 1 este cuatrimestre. Para nosotros, fue el primer proyecto relacionado a crear un juego o un programa que va más allá de la consola y a los tres nos pareció muy interesante el camino de realización del trabajo practico.